MyApp.java

```
import java.util.ResourceBundle;
public class MyApp
{
public int userlogin(string inuser,string inpwd)
{
ResourceBundle rb= ResourceBundle.getBundle(“config”);
String username=rb.getstring(“username”);
String password=rb.getstring(“password”);
If(inuser.equals(username)&& inpassword(password))
    return 1;
else
    return 0;
}
```

Config.properties

```
username=abc
password=abc@1234
```

MyAppTest.java code

```
package myproject;
import org.testng.Assert;
import org.testng.annotations.Test;

import com.myproject.app;

public class apptest {
@Test
public void testlogin1()
{
app myapp=new app();
Assert.assertEquals(0,myapp.userlogin("abc","abc1234"));
}
@Test
public void testlogin2()
{
app myapp=new app();
Assert.assertEquals(1,myapp.userlogin("abc","abc@1234"));
}
}
```

```
<dependencies>
 <!-- https://mvnrepository.com/artifact/org.testng/testng -->
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.5.1</version>
  <scope>test</scope>
</dependency>
 </dependencies>
```

app1/app.py(File)

```
from flask import Flask
app = Flask(__name__)
@app.route('/')
def hello():
return "Hello from App 1!"
if __name__ == '__main__':
app.run(host='0.0.0.0', port=5000)
```

app1/requirements.txt (file)

```
flask==3.0.0
```

app1/Dockerfile (file)

```
FROM python:3.12-slim
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

app2/app.py (file)

```
import requests
response = requests.get("http://app1:5000/")
print("Response from App 1:", response.text)
```

app2/requirements.txt (file)

```
requests==2.31.0
```

app2/Dockerfile(file)

```
FROM python:3.12-slim
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

docker-compose.yml

```
version: '3.9'
services:
  app1:
    build: ./app1
   networks:
      - app-network
   ports:
       - "5000:5000"
 app2:
    build: ./app2
    networks:
       - app-network
    depends_on:
       - app1
networks:
   app-network:
        driver: bridge
```

Command —>>>>>docker-compose build then docker-compose up

{Folder->App1}{file->app.py}

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hw-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hw-container
          image: <name>/app1
          ports:
            - containerPort: 5000
```

Kubernetes Service (service.yam0l)

```
apiVersion: v1
kind: Service
metadata:
  name: hello-world
spec:
  type: NodePort
  selector:
    app: hello-world
  ports:
    - port: 5000
      targetPort: 5000
```

```
docker build -t <name>/app1 .
docker push <name>/app1
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
Kubectl get pods
Kubectl scale deployment/hw-deployment –replicas=3
Kubectl get pods
Kubectl get deployment
Kubectl port-forward svc/hello-world 5000:5000
```